

1. 구조 확립

주제 및 확장에 대한 아이디어 얻음

[참고] 기능 확장 로드맵 (1차~4차) - 성적 관리 프로그램

기존과제 참고: 이 표를 참고하여 자신의 주제를 정의하고, 최종 메뉴를 기획 (4단계의 확장 고려)

차수	과제 목표 및 주요 내용	핵심 프로그래밍 기술 및 개념	화면 설계
1차 과제	기초 입출력 및 산술 연산 • 사용자 정보(이름, 점수 등) 입력받기 • 현재 평균 계산 및 지난 학기와의 차이 산출하기	• 입출력: input(), print() • 할당값: =, +=, -=, float() • 산술 연산: 덧셈, 시프트, 뺄셈 • 출력 포매팅: f-string 활용	
2차 과제	리스트 자료구조 활용 & 조건문 • 데이터를 저장할 수 있는 리스트에 저장 • 리스트 복사/제거/삽입 • 리스트 복사/제거/삽입	• 리스트 조작: append(), insert() • 리스트 복사: deepcopy(), copy() • 리스트 삭제: del, pop(), remove() • 조건문: if, elif, else (복합 문장)	
3차 과제	제어문(반복/조건) 및 메뉴 시스템 • 메뉴 선택에 따른 기능 실행 • 전체 리스트 순회 및 조건별 출력하기	• 무한 반복: while True: break • 반복문: for 문, range 사용, 리스트 순회	
4차 과제	함수 모듈화 및 디버거의 전환 • 기능별 함수 정의 및 모듈(import) 활용 • 리스트 구조를 재배치할 수 있어야 가능함	• 함수: def 키워드(parameter) 반환값 • 모듈화: import 키워드 활용 • 자료구조: 코드블록 -> 디버거(IDE)	

githubreadme (2).pdf
PDF

코드 및 실행 화면 (2).pdf
PDF

2차 과제에 맞게 내가 만든 프로그램을 확장해야해. 아이디어를 줄래?
코드도 부탁해. (참고한 pdf는 내가 1차 과제까지 진전시킨 내용이야,
참고해줘)

몇 초 동안 생각함 >

좋아. 네 1차 주제는 “나만의 음악 취향 분석 및 플레이리스트 관리 시스템”이고, v1.0에서는 곡 제목/아티스트/장르/재생 횟수/평점을 입력받아 선호도 점수를 계산했어. 실제 1차 코드도 `input()`, `int()`, `float()`, f-string 중심으로 구성되어 있었고.

2차 과제는 여기서 리스트 + 조건문을 추가하면 돼.

2차 확장 아이디어

기존 1차는 곡 1개만 입력했지?

2차에서는 곡 여러 개를 리스트에 저장하고, 선호도 점수에 따라 등급을 나누면 좋아.

추천 확장 방향:

- 곡 제목, 아티스트, 장르, 재생 횟수, 평점을 각각 리스트에 저장
- append()로 곡 정보 추가
- insert()로 추천곡 하나를 중간에 삽입
- 리스트 복사로 백업 리스트 만들기
- if-elif-else로 등급 판별
 - 90점 이상: 인생곡
 - 70점 이상: 추천곡
 - 50점 이상: 일반곡
 - 50점 미만: 정리대상

main.py 코드

insert, append, 리스트 복사 이 세가지 문법이 들어가도록 확산시키는
내용을 몇개 추천해줄래?

좋아. 네 음악 플레이리스트 주제에서 `append()`, `insert()`, `리스트 복사` 가 자연스럽게 들어가는 확장 아이디어는 이런 것들이 좋아.

1. 기본 플레이리스트 + 추천 플레이리스트 만들기

내용:

사용자가 입력한 곡은 `append()` 로 기본 플레이리스트에 추가한다.

기본 플레이리스트를 `copy()` 로 복사해서 추천 플레이리스트를 만든다.

AI 추천곡은 사용자가 가장 좋아하는 곡 뒤에 `insert()` 로 삽입한다.

가장 추천. 과제 의도에 제일 잘 맞춤.

2. 원본 플레이리스트 + 수정용 플레이리스트 비교

내용:

사용자가 입력한 곡들을 `append()` 로 저장한다.

원본을 보존하기 위해 리스트를 복사한다.

복사한 리스트에서 "정리대상" 곡 앞에 "교체 추천곡"을 `insert()` 로 추가한다.

의미:

원본은 그대로 두고, 수정된 플레이리스트를 따로 만  : 구조라서 리스트 복사의 이유가 분명함.

2. 코드 참고 및 세부내용 수정

리스트 복사 부분을 자연스럽게 녹이기 위해 AI의 조언을 얻음

2601_4주차_리스트개념과 메소드.pdf PDF

여기서 말하는 복사는 copy를 이용하는게 아닐거야 . pdf를 참고해서 수정해줘

4s 동안 생각함 >

맞아. 교수님 PDF 기준으로 "리스트 복사"는 `copy()` 보다 아래 두 방식이 핵심이야.

```
Python
복사본 = list(원본리스트)
복사본 = 원본리스트[:]
```

PDF에서도 단순할당 `values = temps` 는 같은 리스트 주소를 가리키는 것이고, 진짜 복사는 `values = list(temps)` 또는 `values = temps[:]` 라고 설명돼 있어. 2601_4주차_리스트개념과 메소드

그래서 코드에서 이 부분을 바꾸면 돼.

(+)

과제용으로는 슬라이싱 배웠으니까 `titles[:]` 가 더 좋아 보여.

수정된 핵심 부분은 이렇게 쓰면 돼.

```
Python
# 4. 리스트 복사: 슬라이싱을 이용한 복사
backup_titles = titles[:]

# 5. insert()를 이용한 추천곡 삽입
titles.insert(1, "AI 추천곡")
artists.insert(1, "AI 추천 아티스트")
genres.insert(1, "Pop")
play_counts.insert(1, 80)
ratings.insert(1, 4.5)
```

출력도 이렇게 유지하면 됨.

```
Python
print("\n==== 리스트 복사 및 삽입 확인 =====")
print(f"추천곡 삽입 전 곡 제목 리스트: {backup_titles}")
print(f"추천곡 삽입 후 곡 제목 리스트: {titles}")
```

즉 최종 설명은 이렇게 적으면 좋아.

"슬라이싱 `[:]` 을 이용하여 기존 곡 제목 리스트를 별도의 리스트로 복사하였다. 이후 `insert()` 를 사용해 AI 추천곡을 1번 인덱스에 삽입하여, 복사 전 리스트와 삽입 후 리스트의 차이를 확인하였다."

이렇게 하면 PDF 내용이랑 딱 맞아.

추가로 AI 추천곡은 상단에 파라미터로 설정

ai 추천곡은 그냥 악류의 소문의 낙원으로 정의해줘 상단에

좋다, 그럼 과제 느낌도 살리고 깔끔하게 “상단에 AI 추천곡을 미리 정의”하는 방식으로 가면 된다.

핵심은 코드 맨 위에 변수로 선언하는 거야.

이렇게 수정하면 됨 (상단 정의)

Python



▶ 실행됨

음악 취향 분석 및 플레이리스트 관리 시스템 v2.0

print("==== 음악 취향 분석 및 플레이리스트 관리 시스템 v2.0 =====")

★ AI 추천곡 (상단에 미리 정의)

AI_TITLE = "소문의 낙원"

AI_ARTIST = "AKMU"

AI_GENRE = "발라드"

AI_PLAY_COUNT = 120

AI_RATING = 4.8

2-3차례에 걸쳐 코드 오류 검증 요구

```
print(f"개인 평점 : {ratings[1]:.1f}점")
print(f"선호도 점수 : {preference_scores[1]:.2f}")
print(f"등급 : {grades[1]}")

print("\n3번 곡")
print(f"곡 제목 : {titles[2]}")
print(f"아티스트 : {artists[2]}")
print(f"장르 : {genres[2]}")
print(f"재생 횟수 : {play_counts[2]}회")
print(f"개인 평점 : {ratings[2]:.1f}점")
print(f"선호도 점수 : {preference_scores[2]:.2f}")
print(f"등급 : {grades[2]}")

# 리스트 복사 및 삽입 확인
print("\n==== 리스트 복사 및 삽입 확인 =====")
print(f"추천곡 삽입 전 곡 제목 리스트 : {backup_titles}")
print(f"추천곡 삽입 후 곡 제목 리스트 : {titles}")

<< 다시 검증해줘
```

2초 동안 생각함

↓

검증 결과 지금 코드는 실행 오류 없이 넘어

최종 코드를 얻음

```
# 파일이름 : main.py
# 작 성 자 : Choi Seo Yeon

# 0. AI 추천곡 정보
AI_TITLE = "소문의 낙원"
AI_ARTIST = "AKMU"
AI_GENRE = "발라드"
AI_PLAY_COUNT = 120
AI_RATING = 4.8

# 1. 빈 리스트 생성
titles = []
artists = []
genres = []
play_counts = []
ratings = []
preference_scores = []
grades = []

# 2. 첫 번째 곡 입력
title = input("첫 번째 곡 제목을 입력하세요: ")
artist = input("아티스트를 입력하세요: ")
genre = input("장르를 입력하세요: ")
play_count = int(input("재생 횟수를 입력하세요: "))
rating = float(input("개인 평점(0~5)을 입력하세요: "))

score = (play_count / 2) + (rating * 10)
```

